Case 1 - Linear Model with Curse of Dimensionality

02582 Computational Data Analysis

s194591 & s225236

March 28, 2026

Introduction/Data description

In this assignment, we are going to work with the task of trying to fit a model to data where we have a limited amount of data, and it is high-dimensional - the *curse of dimensionality*. The task is hard because when you have limited data but high dimensionality most complex models tend to overfit the noise instead of fitting to the data correctly. We are then tasked with using this model to predict another 1000 points and estimate the error this model may have on these 1000 predictions.

The dataset $\mathcal{D}$ is a collection of points $\left\{y^{(i)}, x_1^{(i)}, \cdots, x_{100}^{(i)}\right\}_{i=1}^{100}$. Furthermore, many of the data points contain corrupted data with missing values for certain data points. When inspecting a visual plot of the estimated distributions (bar plots) of the different features over the dataset, it is clear that feature 96 to feature 100 are categorical, and that feature 97 is a constant value. We will discuss how to tackle this in *Factor Handling*.

To tackle the problem we will look at different models, to demonstrate how they might fail in this scenario, analyze the reasons behind these failures, and ultimately identify the optimal model we might find with the tools we have learned so far.

Model and method

For this section we are considering models which are linear with respect to the parameters. In short, with a feature expansion $\phi(\mathbf{x})$ we seek a model on the form of

$$f(\mathbf{x}) = \theta^\top \phi(\mathbf{x}) = \sum_{i=0}^{100} \theta_i \cdot \phi_i(x_i), \tag{1}$$

and we will look at ordinary least squares as well as a combination of Ridge Regression with Lasso Regression in the form of ElasticNet, optimized via the LASSO algorithm.

OLS is the simplest form, as it seeks to find a solution to the residual sum of squares

$$\arg\min_\theta \mathcal{L} = \arg\min \sum_n \left(y^{(n)} - f(\mathbf{x}^{(n)})\right)^2. \tag{2}$$

where $\mathbf{x}^{(n)}$ is the dataset entries in $\mathcal{D}$. When the derivative is taken with regards to the weights, and set to 0, it forms the solutions as

$$\hat{\theta}_{\text{OLS}} = \left(\mathbf{X}^\top \mathbf{X}\right)^{-1} \mathbf{X}^\top \mathbf{y}. \tag{3}$$

Ridge Regression adds a penalty term to the Residual Sum of Squares in equation (2) based on the magnitude of the parameter values

$$\arg\min_{\theta} \mathcal{L} = \arg\min \sum_{n} \left(y^{(n)} - f(\mathbf{x}^{(n)}) \right)^2 + \lambda ||\theta||_2^2, \tag{4}$$

which yields a similar but different solution set for θ

$$\hat{\theta}_{\text{Ridge}} = \left(\mathbf{X}^\top \mathbf{X} - \lambda \mathbf{I} \right)^{-1} \mathbf{X}^\top \mathbf{y}. \tag{5}$$

Lasso Regression has a similar penalty term, but it is based on the absolute values of the theta parameters

$$\arg\min_{\theta} \mathcal{L} = \arg\min \sum_{n} \left(y^{(n)} - f(\mathbf{x}^{(n)}) \right)^2 + \lambda ||\theta||_1. \tag{6}$$

Unfortunately, no closed form solution is readily available for Lasso regression, and instead one must use iterative optimizers like Least-Angle Regression Selection (LARS) to find a solution. Ridge Regression allows for a trade-off between variance and bias. When λ is high, we have low variance but high bias, but when alpha is small, we have low bias with high variance.

ElasticNet combines the Lasso and Ridge Regression models into a combined one, where a term α denotes the trade-off between ridge and lasso penalty terms:

$$\arg\min_{\theta} \mathcal{L} = \arg\min \sum_{n} \left(y^{(n)} - f(\mathbf{x}^{(n)}) \right)^2 + \lambda \left(\frac{1-\alpha}{2} ||\theta||_2^2 + \alpha ||\theta||_1 \right), \tag{7}$$

where $\alpha = 1$ performs pure Lasso regression and $\alpha = 0$ performs pure Ridge Regression; when α is in the region between the two, it is called the *elastic region*. Because Lasso has no closed-form solution, ElasticNet lacks one as well.

Model selection

We will in this section go over the different choices we have to make regarding both data selection and model validation and selection. As we are working with few data points containing corrupt data and high dimensionality, there is some choices that are important to ensure we get a fair model for estimation.

Missing values

Investigating the data, we find that many observations have missing features. Since we deal with a number of features $\sim$ number of observations, we cannot simply remove observations that have missing values. This leads us to methods which replace those, with values inferred from the training set.

Looking at the data plots, we considered all the continuous variables $x_1, \cdots, x_{95}$ to be normally distributed. With this assumption, we estimated the mean and standard deviation for each feature, and drew random values from those estimated distributions. Compared to methods where all NaN values are replaced with mean/median/mode, ours has the major upside, which is preserving the distributions of the features. Furthermore, this approach doesn't skew the data towards the middle values that would act as replacements.

For the categorical variables - feature 96 through 100 - the data exhibits the behaviour that if one categorical value is missing, then so are all the others. This can simplify the handling of this data, as we can simply set all the values of these to 0. More on this in *Factor Handling*.

Factor Handling

Many models will assume your data is in a continuous space, but what happens if your data does not adhere to this. In this case, the last 5 features of the data exhibit this kind of categorical nature, and feature number 97 only takes on a constant value. This can all be seen in figure 1, where a barplot of the last 8 features are shown.

In our case, the 5 categorical features will be one-hot encoded. A one-hot encoding is beneficial in this case, as the difference between two categorical variables can't be expressed in euclidean distance between them, as the models might.

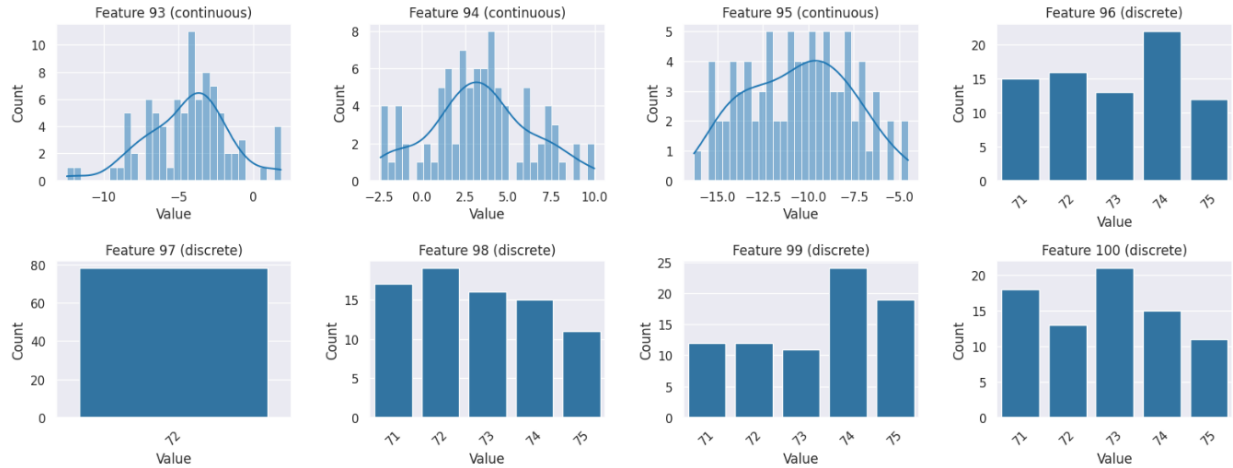


Figure 1: Data Distribution estimation as seen for the tailing 8 features. Feature 96-100 are categorical, with feature 97 constant. Feature 1 to 92 all exhibit continuous nature of their distributions, and mostly Gaussian looking.

As explained in *missing values*, when one categorical variable is missing its values, so are all the rest. For this we simply avoid using these for model predictions by having the 0 vector. This essentially models it as an implicit "missing". The total number of features in the training data with this approach is then:

$$100 - 5 + 1 + 4 \cdot 5 = 116, \tag{8}$$

so the last 21 features are corresponding now to the categorical variables.

Model validation

We go over the different methods used to validate the models we come fouth with. Model validations were done on independent hold out tests sets.

Repeated Cross Validation: In order to validate models and select optimal parameters, we opt to do repeated cross-fold validations. This is a necessary step to forego underestimating the generalization capabilities of the model, otherwise known as an optimistic verdict. To get a good estimate of the true error, we do 10 repeated folds of 10 fold cross validation. This means we train on 90 data points and test on 10 data points for a total of 100 folds. This gives a lower variance estimate of the standard error.

With this approach, we estimate with the model from the 1SE rule the RMSE to be around 40.7. If we calculate among the Cross Validations the *standard error* and *standard deviation* on the hold out tests sets, then in figure 3a and figure 3b we see a confidence interval over the errors. We summarize these below

$$\mathrm{CI}_{\mathrm{SE}} : [38.86, 42.60], \quad \mathrm{CI}_{\mathrm{SD}} : [22.04, 59.43], \tag{9}$$

where the standard error gives a much narrower confidence interval than the one based on the standard deviation. This is because

$$\mathrm{SE} = \frac{\sigma}{\sqrt{n}}, \tag{10}$$

where n here is the number of folds. This in practice means the confidence in our estimate of the error narrows as we estimate it independently more times, which is reflected in confidence interval computations.

Bootstrapping: Is another strong tool which can be helpful when you have data that has distributions different to those that may be assumed for certain tools. Bootstrapping implies sampling your dataset with replacement many times, and perform runs on these, to then in the end infer statistical properties.

We did not opt to use bootstrapping, but wanted to mention the possibility.

KLIEP We also attempted to implement KLIEP method, which estimates the RMSE by using the ratio between train and test densities, and scaling CV residuals appropriately.[1]

$$w = \frac{p_{test}}{p_{train}}, \tag{11}$$

$$R\hat{M}SE \approx \sqrt{\frac{\sum_{i=0}^{100} w(x_i) \cdot (y_i - \hat{y}_i^{-i})^2}{\sum_{i=0}^{100} w(x_i)}}, \tag{12}$$

Where $\hat{y}_i^{-i}$ is the prediction at the left out points of cross validation. This way it up-weighs the validation errors of test regions that are under-represented in the training data but highly prevalent in the test data. This may give a less biased RMSE, as we take more uncertainty into account.

Results

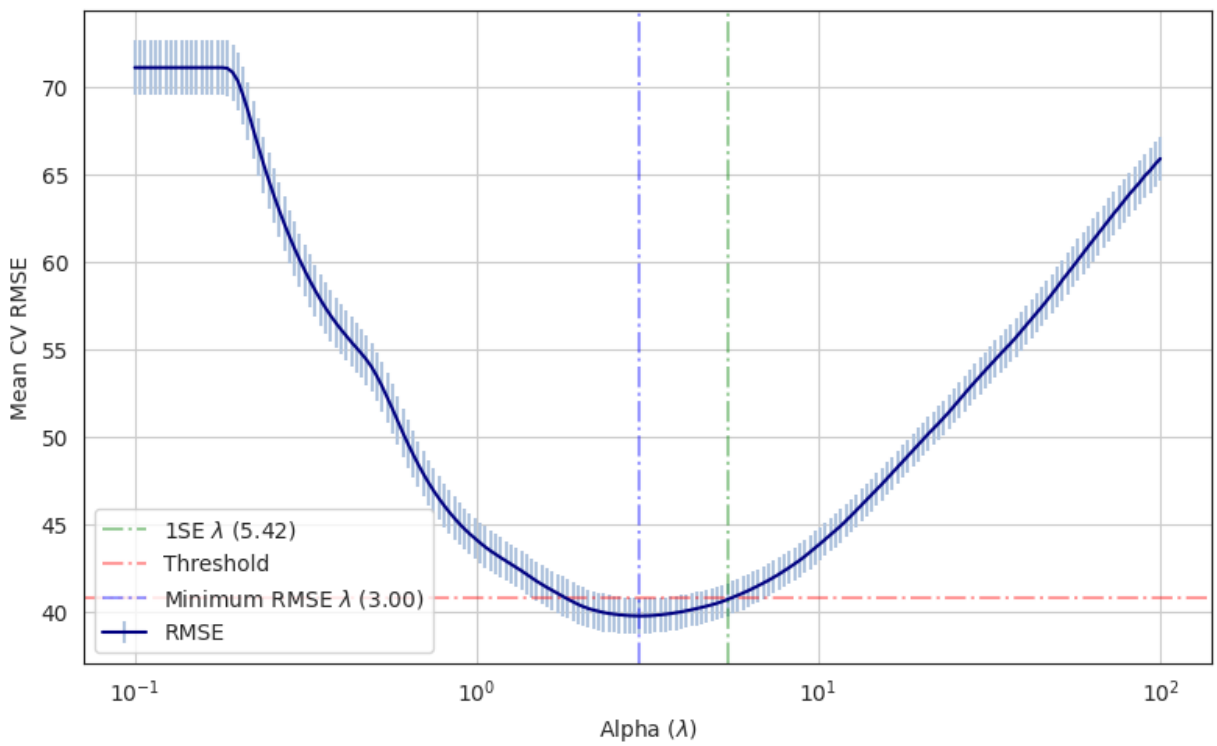


Figure 2: Cross Validation over the different values of λ. Cross validation is also done over parameter α for ElasticNet

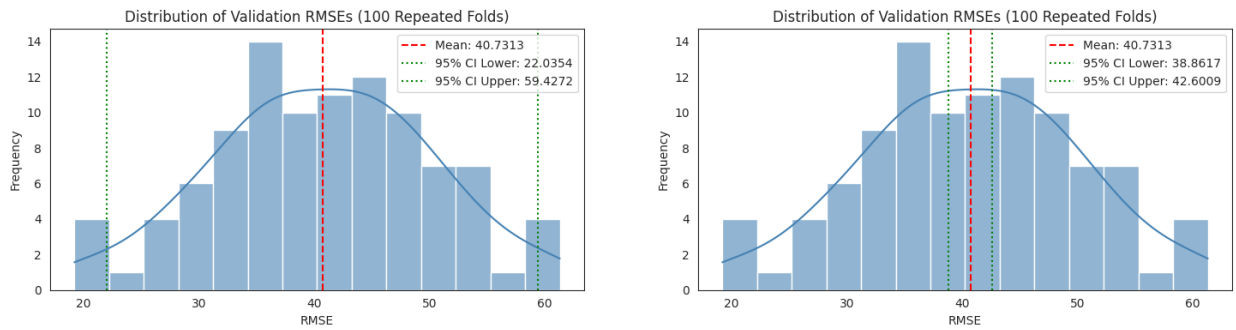
We ran repeated cross validation with 10 splits and 10 runs, for a total of 100 folds; this was for different values of regularization λ and trade-off between L2 and L1 loss α. In figure 2 we see how the RMSE error changes as we navigate the parameter space for λ at found best $\alpha = 1$ value for ElasticNet, meaning we are in a pure Lasso model. As we can see, a λ value of around 3 was found at the optimum. Our best performing model was an ElasticNet model, with the largest regression parameter within 1 SE from the optimum, to obtain the simplest model within the standard error, described by eq. (7), with parameters:

$$\alpha = 1, \lambda_{1_\text{SE}} = 5.42$$

which is a pure Lasso model. We wanted to estimate the RMSE we would get on unknown data, and we found that the final model only kept 16 features. More specifically it kept the following features:

$$[x_5, x_9, x_{22}, x_{23}, x_{28}, x_{30}, x_{31}, x_{35}, x_{44}, x_{48}, x_{50}, x_{61}, x_{63}, x_{64}, x_{73}, x_{80}]$$

So a lot of features were estimated to be not as relevant for the problem at hand. There is a range of RMSE over the 100 repeated folds for the final model, however, and we seem them in figure 3. We see that the CI for the RMSE error using the standard deviation is quite wide, but with SE we get a narrower CI.



(a) 1 Standard Deviation CI

(b) 1 Standard Error CI

Figure 3: Confidence Intervals (CI) on RMSE on the hold out validation sets. Left: 1 Standard Deviation. Right: 1 Standard Error.

Use of AI declaration

For our project we have decided to use gen AI (ChatGPT & Gemini) to help with code structure and debugging the code. It was also used as a research tool, to find and describe methods of estimating RMSE without test values, from which we found then and took inspiration to use KLIEP method. All findings were reviewed by the two of us, before inclusion in the project.

References

[1] Masashi Sugiyama et al. "Direct importance estimation for covariate shift adaptation". In: *Annals of the Institute of Statistical Mathematics* 60 (2008), pp. 699–746. DOI: 10.1007/s10463-008-0197-x. URL: https://doi.org/10.1007/s10463-008-0197-x.